

Warden — Internal Process Manager

A self-hosted, PM2-equivalent process manager built in TypeScript for Node 18+.

Manages **any** process (Node.js, Python, Ruby, binaries, shell scripts) with the full PM2 feature set and first-class telemetry.

Features

Category	Feature
Process modes	Fork mode (any process), Node.js Cluster mode (multi-instance, port-sharing)
Auto-restart	Crash detection, exponential backoff, <code>max_restarts</code> , <code>min_uptime</code> stability gate
Zero-downtime reload	Rolling reload for cluster apps (<code>warden reload</code>)
Scale	<code>warden scale <name> <n></code> — add/remove cluster workers at runtime
Watch	File-system watch triggers reload (<code>--watch</code>)
Cron restart	Scheduled restarts via cron expression (<code>--cron "0 0 * * *"</code>)
Memory limit	Auto-restart on memory threshold (<code>--max-memory-restart 150M</code>)
Log management	Per-process stdout/stderr files, rotation, streaming via <code>warden logs</code>
Persistence	<code>warden save</code> / <code>warden resurrect</code> with <code>~/.warden/dump.json</code>
Startup	Generate systemd / launchd service via <code>warden startup</code>
Ecosystem config	Declare multiple apps in <code>ecosystem.config.js</code> / <code>.json</code>
Live dashboard	<code>warden monit</code> — auto-refreshing terminal table
HTTP API	<code>GET /processes</code> , <code>/summary</code> , <code>/processes/:id/metrics</code>
Prometheus	<code>GET /metrics</code> endpoint (prom-client gauges/counters)
Telemetry	SQLite time-series, 24-hour retention, 5-second sampling

Installation

```
# From npm (global CLI):
npm install -g @roshan-gamage/warden

# From source:
git clone https://github.com/RoshanGamage01/warden.git
cd warden
npm install
npm run build
npm link # optional: link `warden` globally
```

The `warden` binary is at `./dist/bin/warden.js` after build.

Linux (Ubuntu/Debian): Prebuilt binaries are included for Node 20–24. If install still compiles from source and fails with `not found: make`, install build tools first:

```
sudo apt-get update && sudo apt-get install -y build-essential python3
```

Quick start

```
# Start any Node script directly
warden start ./server.js --name api

# Start a Python script
warden start ./worker.py --interpreter python3 --name worker

# Start with 4 cluster instances (Node.js only, port-sharing)
warden start ./app.js --instances 4 --name app

# Use an ecosystem config (ecosystem.config.js, warden.config.json, or *.config.js)
warden start ecosystem.config.js

# List all processes
warden list

# Stream logs
warden logs           # all processes
warden logs api       # by name
warden logs 0         # by id

# Live dashboard
warden monit
```

CLI Reference

Process management

Command	Description
<code>warden start <script></code>	Start a process (<code>.js</code> , <code>.py</code> , binaries, etc.)
<code>warden start <ecosystem.config.js></code>	Start all apps in an ecosystem config
<code>warden stop <id name></code>	Stop a process
<code>warden restart <id name></code>	Restart a process
<code>warden reload <id name></code>	Zero-downtime reload (cluster) or graceful restart (fork)
<code>warden delete <id name></code>	Stop and remove from registry
<code>warden scale <id name> <n></code>	Scale cluster app to N workers
<code>warden reset <id name></code>	Reset restart counters

Inspection

Command	Description
<code>warden list / warden ls</code>	Process table with CPU/mem/uptime
<code>warden describe <id name></code>	Full process detail including cluster workers
<code>warden monit</code>	Live refreshing dashboard (Ctrl+C to quit)
<code>warden ping</code>	Check daemon status

Logs

Command	Description
<code>warden logs [id]</code>	Stream logs (Ctrl+C to stop)
<code>warden logs -n 50 [id]</code>	Show last 50 historical lines then stream
<code>warden flush [id]</code>	Truncate log files

Persistence & startup

Command	Description
<code>warden save</code>	Write current process list to <code>~/.warden/dump.json</code>
<code>warden resurrect</code>	Start all saved processes
<code>warden startup [platform]</code>	Install system startup service (auto, systemd, launchd)
<code>warden unstartup [platform]</code>	Remove startup service

Daemon

Command	Description
<code>warden kill</code>	Gracefully stop all processes and the daemon

start options

```
-n, --name <name>           App name (defaults to script filename)
-i, --instances <n|max>     Number of instances; "max" uses all CPUs
    --exec-mode <fork|cluster> Execution mode (auto-set to cluster if -i > 1)
    --interpreter <binary>   Interpreter (e.g. python3, ruby)
    --node-args <args>       Extra Node.js args
    --cwd <dir>              Working directory
-e, --env <KEY=VAL>         Environment variable (repeatable)
    --watch                  Restart on file changes
    --max-memory-restart <size> Restart threshold (e.g. 150M, 1G)
    --cron <expr>           Scheduled restart (cron expression)
    --max-restarts <n>       Max crash-restart attempts (default 16)
    --min-uptime <time>     Stability gate (e.g. 1s, 500ms)
    --restart-delay <ms>    Delay between restarts
    --no-auto-restart        Disable crash auto-restart
    --merge-logs             Merge stdout and stderr into one file
    --kill-timeout <ms>     Graceful shutdown timeout
```

Ecosystem config

```
// ecosystem.config.js
module.exports = {
  apps: [
    {
      name: 'api',
      script: './src/api.js',
      instances: 2,
      exec_mode: 'cluster',
      env: { NODE_ENV: 'production', PORT: '3000' },
      max_memory_restart: '500M',
    },
    {
      name: 'worker',
      script: './src/worker.js',
      watch: ['./src'],
      ignore_watch: ['node_modules'],
      cron_restart: '0 3 * * *', // restart daily at 3am
    },
    {
      name: 'ml-model',
      script: './predict.py',
      interpreter: 'python3',
      autorestart: true,
      max_restarts: 5,
    },
  ],
};
```

```
warden start ecosystem.config.js
```

HTTP API

Runs on `http://127.0.0.1:9615` by default.

Endpoint	Description
<code>GET /health</code>	<code>{ status: 'ok', pid }</code>
<code>GET /processes</code>	All processes with latest CPU/mem
<code>GET /processes/:id</code>	Single process + 120 recent metric points
<code>GET /processes/:id/metrics?range=1h</code>	Time-series data (range: 1h , 6h , 24h , 7d)
<code>GET /summary</code>	Aggregate totals (online, stopped, errored, CPU, memory)
<code>GET /metrics</code>	Prometheus exposition format

Prometheus / Grafana

Point your Prometheus scrape config at `http://127.0.0.1:9615/metrics` :

```
# prometheus.yml
scrape_configs:
  - job_name: warden
    static_configs:
      - targets: ['127.0.0.1:9615']
```

Available metrics:

- `warden_process_cpu_percent{warden_id, name}`
 - `warden_process_memory_bytes{warden_id, name}`
 - `warden_process_restarts_total{warden_id, name}`
 - `warden_process_uptime_seconds{warden_id, name}`
 - `warden_process_online{warden_id, name, status}`
 - Standard Node.js process metrics via `prom-client`
-

File locations

Path	Purpose
<code>~/.warden/daemon.sock</code>	Unix domain socket (IPC)
<code>~/.warden/daemon.pid</code>	Daemon PID file
<code>~/.warden/dump.json</code>	Saved process list
<code>~/.warden/metrics.db</code>	SQLite metrics database (24-hour rolling window)
<code>~/.warden/logs/<name>--<id>/out.log</code>	Stdout log
<code>~/.warden/logs/<name>--<id>/err.log</code>	Stderr log
<code>~/.warden/logs/<name>--<id>/out.log.1</code>	Rotated log (up to 5 backups)

Architecture overview

```
CLI (warden)
├─ JSON-RPC over Unix domain socket
└─
  Daemon (God process)
  ├── IPC Server      ─ RPC request router
  ├── AppRegistry     ─ Registry of all managed apps
  │   ├── ProcessSupervisor ─ fork-mode lifecycle, backoff, log piping
  │   └── ClusterManager  ─ fork(cluster/master.js) → N workers
  ├── WatchManager    ─ chokidar → restart
  ├── CronScheduler   ─ croner → restart
  ├── MemoryMonitor   ─ pidusage → restart on threshold
  ├── MetricsSampler  ─ pidusage every 5s → SQLite
  ├── LogRotator       ─ size-based rotation every 60s
  └── HTTP API (Fastify) ─ /processes, /metrics (Prometheus)
```

Daemon bootstrap

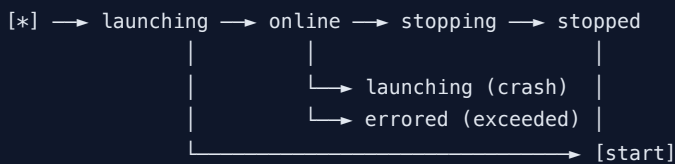
1. CLI checks if `~/.warden/daemon.sock` is connectable
2. If not, spawns daemon detached: `node dist/daemon/index.js`
3. CLI polls socket every 100 ms (max 5 s) for readiness
4. CLI connects and sends JSON-RPC

IPC framing

```
[4-byte BE uint32: body length] [UTF-8 JSON payload]
```

Handles TCP fragmentation and multiple messages per chunk.

Process state machine



Development

```
npm run dev      # watch mode build
npm test         # unit tests
npm run test:watch # watch mode

# Run integration tests (starts/stops real daemon)
npm test         # integration tests are auto-run (uses isolated ~/.warden)

# Skip integration tests
SKIP_INTEGRATION=1 npm test
```

Windows support

On Windows, the Unix domain socket path is replaced with a named pipe (`\\.\pipe\warden-daemon`). All other functionality is identical. The `pidusage` and `better-sqlite3` packages are cross-platform. If prebuilt binaries are not available for your Node version, native compilation may be required — on Windows use [Build Tools for Visual Studio](#); on Linux use `build-essential` (see Installation above).

Environment variables

Variable	Default	Description
<code>WARDEN_API_PORT</code>	<code>9615</code>	HTTP API port
<code>WARDEN_API_HOST</code>	<code>127.0.0.1</code>	HTTP API bind address

License

MIT